Before You Leave: AI-Powered Personal Belongings Detection System

AI-Based Personal Item Detection and Checklist Verification · "Check your belongings before you leave."

Still-Image Prototype · Proposed, not real-time · No experimental results yet

Team

Panyasiri Kiattiphumvarakun Student ID::116710905066-2
Phuwanai Khuyjad Student ID::116710905064-7
Worawut Phuangmalai Student ID:116710905077-9
Group Name: [Scuba]

"Leave nothing behind. Let AI help you double-check."

Problem & target users

Students forget belongings on classroom desks, in libraries, or in dorms; manual checks are inconsistent, and small or partly hidden objects get overlooked. Consequences: unnecessary travel, wasted time, inconvenience, extra expenses. Target users: university students, office workers, anyone verifying belongings before leaving a room.



Forgotten Personal Items → AI-Based Detection → Checklist Verification → Reminder Before Leaving

Objectives & workflow

Detect personal belongings in an image using AI and Computer Vision. Compare detected objects against a user-defined checklist; warn when a required item is not detected; show detection results and checklist statuses in an intuitive interface.
Image Input: user uploads a still image of a desk or indoor area → Object Detection: model identifies supported objects and locations → Checklist Comparison: is each required item among detections? → Result Display: detected items, warnings, statuses.

Computer Vision

Primary task: object detection with bounding boxes, class labels, and confidence scores on still images. YOLO (You Only Look Once) is a candidate model to evaluate, not a confirmed final choice.



Checklist logic: Found / Not Detected. Not Detected does not mean absent: an item may be outside the frame, obscured, or missed by the model. Any confidence scores are "mockup values, not experimental findings".

Image data & proposed stack

Planned data: self-collected images of real belongings in desk or indoor settings; public datasets such as COCO where categories exist; extra annotation where classes are missing or underrepresented. Training Set trains the model; Validation Set monitors performance and guides model selection; Test Set evaluates on separate images. Split ratios and dataset size are to be determined by the experiment. Proposed stack: Python (main language) · OpenCV (image processing) · Ultralytics YOLO (candidate detection framework) · Google Colab (training and experimentation) · Roboflow (dataset management and annotation) · Streamlit (proposed web prototype interface).
Data Collection → Annotation → Model Training → Model Evaluation → Prototype Development

Expected interface

UI MOCKUP — ILLUSTRATIVE EXAMPLE, NOT ACTUAL TEST RESULTS
Dashboard: uploaded image preview with bounding boxes, checklist table, warning banner. Fictional interface example, not model output.

Checklist	Status
Phone	✓ Found
Wallet	✓ Found
Keys	Not Detected
Student ID	✓ Found

⚠ Keys were not detected in the image. Please check before leaving the room. Status shown by icon and text as well as color.

Evaluation & limitations

Planned metrics: Precision (share of detections that are correct), Recall (share of relevant objects detected), mAP (mean Average Precision across classes and thresholds), Inference Time (per-image processing time under defined test conditions). Set target criteria before experimenting; report numeric results only after experiments are run. Planned testing on unseen images varying lighting, camera angle, object position and distance, background, and partial occlusion; record errors and failure cases. Limitations: occluded objects may be missed; objects outside the frame cannot be detected; performance varies with lighting, angle, image quality, and appearance; Not Detected does not confirm an item was forgotten; users should verify belongings manually.